

2.6 사이킷런으로 수행하는 타이타닉 생존자 예측

목차

1. 실습 개요와 데이터 소개
 1. 타이타닉 생존자 예측 문제
 2. 컬럼 구성
 3. 라이브러리 임포트와 데이터 로딩
2. 데이터 탐색과 결측치 처리
 1. info()로 데이터 확인
 2. 결측치 대체
 3. 문자열 피처의 값 분포 확인
 4. Cabin 속성 가공
3. 탐색적 데이터 분석 - 생존 요인 시각화
 1. 성별에 따른 생존 확률
 2. 객실 등급과 성별의 결합 분석
 3. 연령대별 생존 확률
4. 레이블 인코딩
5. 전처리 함수 통합 - 재사용 가능한 파이프라인
6. 모델 학습과 예측
 1. 피처와 레이블 분리
 2. 학습/테스트 데이터 분리
 3. 3개 알고리즘 비교
7. 교차 검증으로 성능 평가
 1. KFold를 이용한 교차 검증
 2. GridSearchCV로 하이퍼 파라미터 튜닝
8. 핵심 요약 및 머신러닝 활용 포인트

1. 실습 개요와 데이터 소개

1.1 타이타닉 생존자 예측 문제

이번 장은 지금까지 배운 판다스 데이터 핸들링, 데이터 전처리, 모델 학습·평가, 교차 검증, 하이퍼 파라미터 튜닝을 모두 결합한 종합 실습입니다. 캐글(Kaggle)의 대표적인 입문 과제인 타이타닉 데이터로 승객의 생존 여부를 예측합니다.

붓꽃 데이터와 달리 타이타닉 데이터는 결측치가 있고 문자열이 섞여 있어 그대로는 학습이 불가능합니다. 즉 실제 데이터가 어떻게 지저분한지, 그리고 어떻게 다듬어야 하는지를 배우는 것이 이번 장의 핵심입니다.

1.2 컬럼 구성

컬럼명	의미	비고
Survived	생존 여부 (0=사망, 1=생존)	예측 대상 레이블(y)
PassengerId	탑승자 일련번호	식별자 → 삭제 대상
Pclass	객실 등급 (1=일등석, 2=이등석, 3=삼등석)	사회·경제적 지위를 반영
Name	탑승자 이름	고유값 → 삭제 대상
Sex	성별 (male, female)	문자열 → 인코딩 필요
Age	나이	결측치 177건
SibSp	함께 탑승한 형제자매·배우자 수	—
Parch	함께 탑승한 부모·자녀 수	—
Ticket	티켓 번호	불규칙한 문자열 → 삭제 대상
Fare	요금	—
Cabin	선실 번호	결측치 687건, 문자열
Embarked	탑승 항구 (C=Cherbourg, Q=Queenstown, S=Southampton)	결측치 2건, 문자열

1.3 라이브러리 임포트와 데이터 로딩

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

titanic_df = pd.read_csv('./titanic_train.csv')
titanic_df.head(3)
```

줄	코드	상세 설명
1~2	import numpy as np / import pandas as pd	수치 연산과 데이터 핸들링을 위한 기본 라이브러리입니다.
3	import matplotlib.pyplot as plt	파이썬의 기본 시각화 라이브러리입니다. 그래프 크기 조정 등에 사용합니다.

4	import seaborn as sns	matplotlib을 기반으로 한 고수준 통계 시각화 라이브러리입니다. 한 줄로 집계와 그래프를 동시에 그릴 수 있습니다.
5	%matplotlib inline	주피터 노트북 매직 커맨드 입니다. 그래프를 별도 창이 아닌 노트북 셀 안에 바로 표시 합니다. 파이썬 문법이 아니므로 .py 스크립트에서는 사용할 수 없습니다 .
7	titanic_df = pd.read_csv('./titanic_train.csv')	CSV를 DataFrame으로 로딩합니다. ./ 는 현재 디렉터리 를 의미하므로, 노트북과 CSV 파일이 같은 폴더에 있어야 합니다.
8	titanic_df.head(3)	상위 3건으로 데이터 구조를 확인합니다. Cabin에 NaN 이 보이는 것이 눈에 띕니다.

▶ 실행 결과

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen...	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. Jo...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss....	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

2. 데이터 탐색과 결측치 처리

2.1 info()로 데이터 확인

```
print('\n ### train 데이터 정보 ### \n')
print(titanic_df.info())
```

줄	코드	상세 설명
2	print(titanic_df.info())	<code>info()</code> 는 컬럼별 Non-Null 건수와 데이터 타입 을 출력합니다. 참고로 <code>info()</code> 는 반환값이 None 이라 화면에 정보를 직접 출력한 뒤 None 이 함께 찍힙니다. <code>titanic_df.info()</code> 만 써도 동일한 결과를 볼 수 있습니다.

▶ 실행 결과

```
### train 데이터 정보 ###

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   PassengerId 891 non-null    int64
 1   Survived    891 non-null    int64
 2   Pclass      891 non-null    int64
 3   Name        891 non-null    object
 4   Sex         891 non-null    object
 5   Age         714 non-null    float64
 6   SibSp       891 non-null    int64
 7   Parch       891 non-null    int64
 8   Ticket      891 non-null    object
 9   Fare        891 non-null    float64
10   Cabin       204 non-null    object
11   Embarked    889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

■ 결과 해석 - 두 가지 문제 발견

- **결측치** : Age 177건, Cabin 687건, Embarked 2건이 비어 있습니다. 사이킷런은 NaN을 허용하지 않으므로 반드시 처리해야 합니다.
- **문자열(object) 타입** : Name, Sex, Ticket, Cabin, Embarked 5개가 문자열입니다. 이 역시 숫자로 인코딩하거나 삭제해야 합니다.

2.2 결측치 대체

```
titanic_df['Age'].fillna(titanic_df['Age'].mean(), inplace=True)
titanic_df['Cabin'].fillna('N', inplace=True)
titanic_df['Embarked'].fillna('N', inplace=True)
print('데이터 세트 Null 값 갯수 ', titanic_df.isnull().sum().sum())
```

줄	코드	상세 설명
1	titanic_df['Age'].fillna(titanic_df['Age'].mean(), inplace=True)	Age의 결측치를 평균 나이(29.699118) 로 대체합니다. 숫자형 컬럼의 가장 일반적인 대체 전략입니다.
2	titanic_df['Cabin'].fillna('N', inplace=True)	Cabin은 문자열이므로 'N'(None의 약자) 이라는 임의의 문자로 대체합니다. 평균 개념이 없기 때문입니다.
3	titanic_df['Embarked'].fillna('N', inplace=True)	Embarked도 마찬가지로 'N'으로 대체합니다. 최빈값인 'S'로 대체하는 방법도 있습니다.
4	titanic_df.isnull().sum().sum()	sum() 이 두 번 쓰인 점에 주목하세요. 첫 번째 sum은 컬럼별 결측 건수 (Series) 를 구하고, 두 번째 sum은 그 값들을 모두 더해 전체 결측 건수 하나의 숫자 로 만듭니다. 결과 0 은 결측치가 완전히 사라졌음을 뜻합니다.

▶ 실행 결과

```
데이터 세트 Null 값 갯수 0
```

⚠ pandas 2.x 필수 확인 - 이 코드는 경고가 발생합니다

위 코드는 **컬럼을 먼저 선택한 뒤 inplace=True를 적용**하는 연쇄 할당(chained assignment) 방식입니다. pandas 2.x에서 다음 경고가 출력됩니다.

```
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

중간에 생성되는 객체가 **복사본**이라 원본에 반영되지 않을 수 있으며, **pandas 3.0에서는 아예 동작하지 않습니다**. 안정화 버전에서는 아래처럼 **결과를 다시 대입**하는 방식으로 작성하세요.

```
titanic_df['Age'] = titanic_df['Age'].fillna(titanic_df['Age'].mean())
titanic_df['Cabin'] = titanic_df['Cabin'].fillna('N')
titanic_df['Embarked'] = titanic_df['Embarked'].fillna('N')
```

2.3 문자열 피처의 값 분포 확인

```
print(' Sex 값 분포 :\n',titanic_df['Sex'].value_counts())
print('\n Cabin 값 분포 :\n',titanic_df['Cabin'].value_counts())
print('\n Embarked 값 분포 :\n',titanic_df['Embarked'].value_counts())
```

줄	코드	상세 설명
1	titanic_df['Sex'].value_counts()	male 577명, female 314명으로 남성이 약 1.8배 많습니다. 값이 2가지뿐이라 인코딩이 간단합니다.
2	titanic_df['Cabin'].value_counts()	대체값 'N'이 687건으로 압도적이고, 나머지는 'C23 C25 C27'처럼 여러 선실이 한 칸에 들어 있거나 고유값이 지나치게 많습니다. 이 상태로는 인코딩해도 쓸모가 없습니다.
3	titanic_df['Embarked'].value_counts()	S 644, C 168, Q 77, N 2입니다. 값이 4가지로 적당해 그대로 인코딩할 수 있습니다.

▶ 실행 결과

```
Sex 값 분포 :
Sex
male      577
female    314
Name: count, dtype: int64

Cabin 값 분포 :
Cabin
N          687
G6          4
C23 C25 C27  4
B96 B98      4
F2           3
D            3
E101         3
C22 C26       3
... (147개 고유값) ...
Name: count, dtype: int64

Embarked 값 분포 :
Embarked
S      644
C      168
Q       77
N        2
Name: count, dtype: int64
```

2.4 Cabin 속성 가공

```
titanic_df['Cabin'] = titanic_df['Cabin'].str[:1]
print(titanic_df['Cabin'].head(3))
```

줄	코드	상세 설명
1	titanic_df['Cabin'].str[:1]	<code>.str</code> 접근자는 Series의 모든 문자열 원소에 문자열 연산을 일괄 적용합니다. <code>[:1]</code> 은 맨 앞 한 글자만 추출합니다. 'C23 C25 C27' → 'C', 'B96 B98' → 'B'가 됩니다.
2	print(titanic_df['Cabin'].head(3))	147개에 달하던 고유값이 9개(N, C, B, D, E, A, F, G, T) 로 크게 줄었습니다.

▶ 실행 결과

```
0    N
1    C
2    N
Name: Cabin, dtype: object

# 가공 후 분포
Cabin
N    687
C     59
B     47
D     33
E     32
A     15
F     13
G      4
T      1
Name: count, dtype: int64
```

■ 왜 첫 글자만 남기는가

선실 번호의 첫 알파벳은 갑판(Deck) 위치를 나타냅니다. 즉 배의 어느 층에 있었는지를 의미하며, 구명보트 접근성과 관련이 있어 생존과 연관될 가능성이 큼니다. 반면 뒤의 숫자는 개별 방 번호라 예측에 도움이 되지 않습니다. 이렇게 **정보의 핵심만 남기고 불필요한 세부를 제거**하는 것이 피쳐 엔지니어링의 좋은 예입니다.

3. 탐색적 데이터 분석 - 생존 요인 시각화

3.1 성별에 따른 생존 확률

```
titanic_df.groupby(['Sex', 'Survived'])['Survived'].count()
```

줄	코드	상세 설명
1	<code>groupby(['Sex', 'Survived'])</code> <code>['Survived'].count()</code>	두 개의 컬럼으로 그룹화합니다. 성별로 나눈 뒤 그 안에서 다시 생존 여부로 나누어 건수를 셉니다. 결과는 인덱스가 두 단계인 멀티 인덱스 Series 입니다.

▶ 실행 결과

```
Sex    Survived
female 0         81
       1        233
male   0        468
       1        109
Name: Survived, dtype: int64
```

■ 결과 해석 - 성별의 압도적 영향

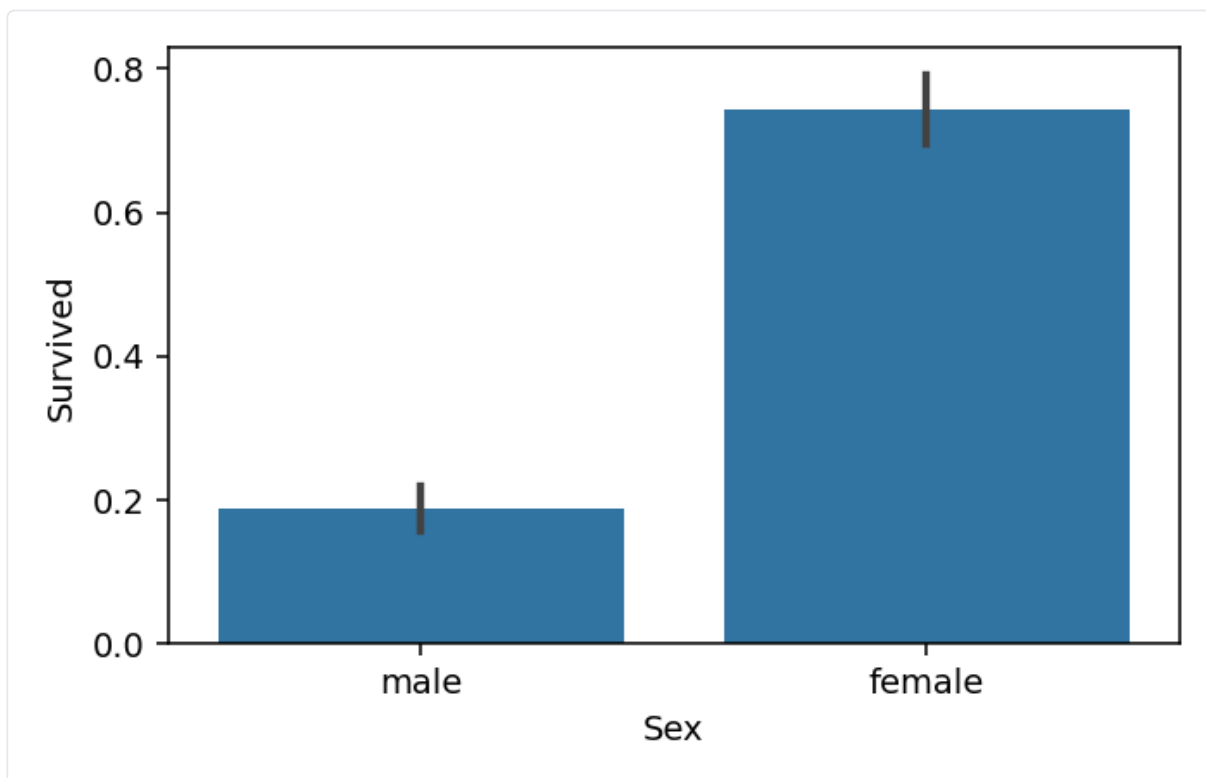
- 여성 : 314명 중 233명 생존 → 생존율 **74.2%**
- 남성 : 577명 중 109명 생존 → 생존율 **18.9%**

약 4배의 차이입니다. "여성과 어린이 먼저"라는 구조 원칙이 실제 데이터에 그대로 반영되어 있으며, **Sex가 가장 강력한 예측 피쳐**임을 알 수 있습니다.

```
sns.barplot(x='Sex', y = 'Survived', data=titanic_df)
```

줄	코드	상세 설명
1	<code>sns.barplot(x='Sex', y='Survived', data=titanic_df)</code>	seaborn의 막대 그래프입니다. x축은 그룹 기준 , y축은 집계할 값 , data 는 대상 DataFrame입니다. 중요한 특징은 barplot이 y값의 평균을 자동으로 계산 한다는 점입니다. Survived가 0/1이므로 평균 = 생존율 이 됩니다. 막대 위의 검은 세로선은 신뢰 구간(오차 범위) 입니다.

▶ 실행 결과



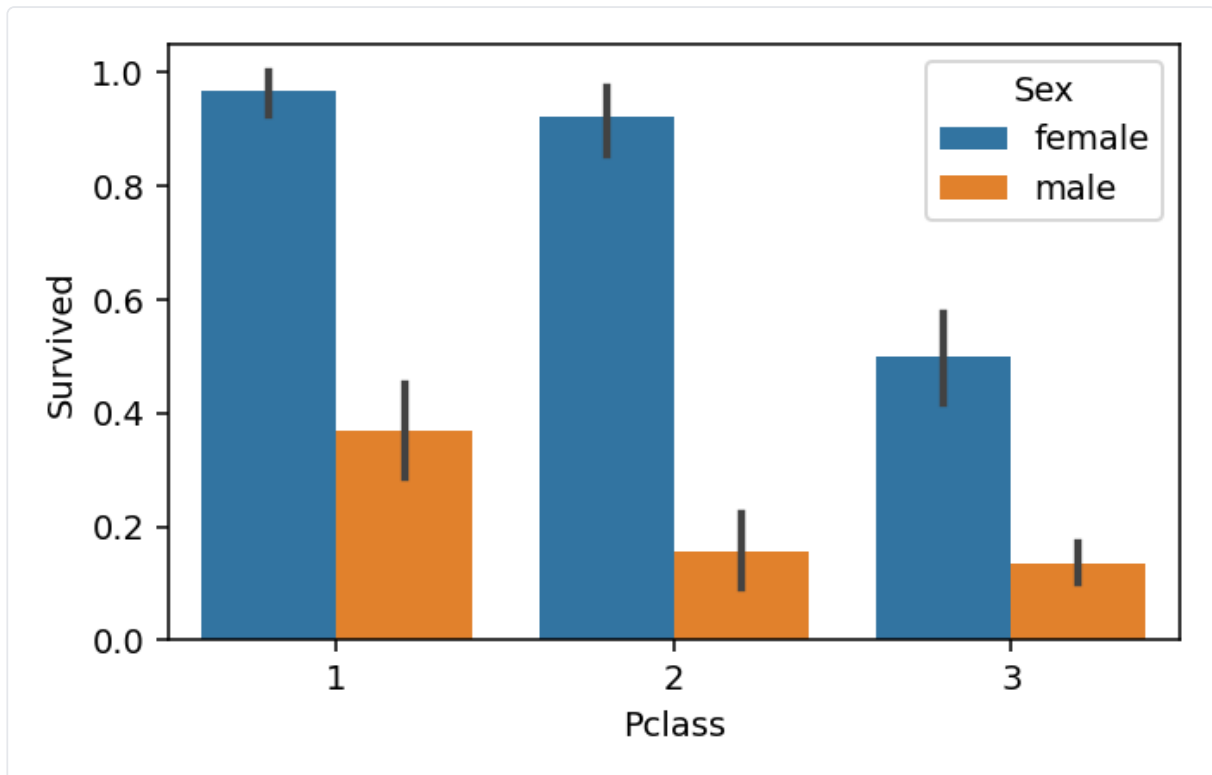
성별에 따른 생존율 — 여성 약 0.74, 남성 약 0.19

3.2 객실 등급과 성별의 결합 분석

```
sns.barplot(x='Pclass', y='Survived', hue='Sex', data=titanic_df)
```

줄	코드	상세 설명
1	<code>sns.barplot(x='Pclass', y='Survived', hue='Sex', data=titanic_df)</code>	hue 파라미터가 핵심입니다. 지정한 컬럼의 값별로 막대를 색깔로 나누어 그립니다. 여기서는 각 객실 등급 안에서 남녀를 나란히 비교할 수 있어, 두 피처의 상호작용 을 한눈에 파악할 수 있습니다.

▶ 실행 결과



객실 등급별·성별 생존율

■ 결과 해석

- 여성은 1등석과 2등석에서 생존율이 거의 100%에 가깝지만, 3등석에서는 약 50%로 급격히 떨어집니다.
- 남성은 1등석이 약 37%, 2·3등석은 15% 내외로 모든 등급에서 낮습니다.
- 주목할 점은 3등석 여성(약 50%)이 1등석 남성(약 37%)보다 생존율이 높다는 것입니다. 즉 성별의 영향이 객실 등급보다 강력합니다.
- 다만 등급도 분명한 영향을 주므로, **Sex와 Pclass 모두 중요한 피처**로 사용해야 합니다.

3.3 연령대별 생존 확률

입력 age에 따라 구분값을 반환하는 함수 설정. DataFrame의 apply lambda식에 사용.

```
def get_category(age):
    cat = ''
    if age <= -1: cat = 'Unknown'
    elif age <= 5: cat = 'Baby'
    elif age <= 12: cat = 'Child'
    elif age <= 18: cat = 'Teenager'
    elif age <= 25: cat = 'Student'
    elif age <= 35: cat = 'Young Adult'
    elif age <= 60: cat = 'Adult'
    else : cat = 'Elderly'

    return cat
```

막대그래프의 크기 figure를 더 크게 설정
plt.figure(figsize=(10,6))

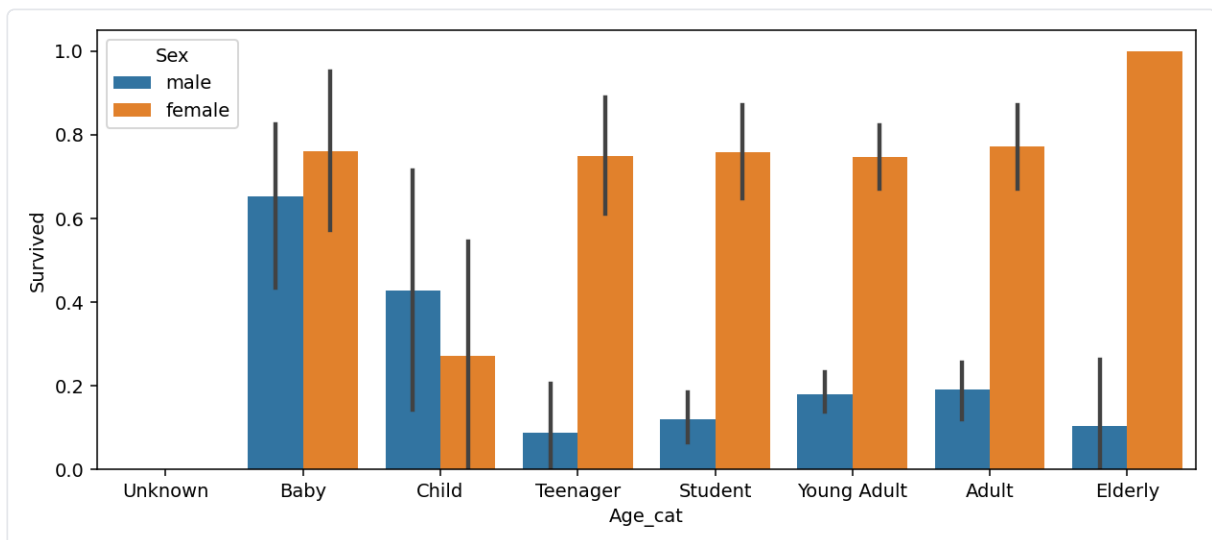
#X축의 값을 순차적으로 표시하기 위한 설정

```
group_names = ['Unknown', 'Baby', 'Child', 'Teenager', 'Student', 'Young Adult', 'Adult', 'Elderly']
```

```
# lambda 식에 위에서 생성한 get_category( ) 함수를 반환값으로 지정.
# get_category(X)는 입력값으로 'Age' 컬럼값을 받아서 해당하는 cat 반환
titanic_df['Age_cat'] = titanic_df['Age'].apply(lambda x : get_category(x))
sns.barplot(x='Age_cat', y='Survived', hue='Sex', data=titanic_df, order=group_names)
titanic_df.drop('Age_cat', axis=1, inplace=True)
```

줄	코드	상세 설명
2~13	def get_category(age): ...	나이를 8개 구간으로 분류하는 함수입니다. <code>if-elif</code> 는 위에서부터 순차 평가 되므로 각 조건에 하한값을 쓸 필요가 없습니다.
5	if age <= -1: cat = 'Unknown'	나이를 -1 등으로 표시해 결측을 나타낸 경우를 위한 분기입니다. 다만 이미 평균값으로 대체했기 때문에 이 조건에 걸리는 데이터는 하나도 없습니다 . 그래서 그래프의 Unknown 구간이 비어 있게 됩니다.
16	plt.figure(figsize=(10,6))	그래프 크기를 가로 10인치, 세로 6인치로 키웁니다. 카테고리가 8개라 기본 크기로는 x축 라벨이 겹칩니다.
19	group_names = ['Unknown', 'Baby', ...]	x축에 표시할 순서를 정의합니다. 지정하지 않으면 데이터 등장 순서나 알파벳 순으로 정렬되어 나이 순서가 뒤죽박죽 이 됩니다.
23	titanic_df['Age_cat'] = titanic_df['Age'].apply(lambda x : get_category(x))	<code>apply</code> 로 각 나이 값에 함수를 적용해 Age_cat 임시 컬럼 을 만듭니다. <code>apply(get_category)</code> 처럼 함수를 직접 넘겨도 동일하게 동작합니다.
24	sns.barplot(..., order=group_names)	<code>order</code> 파라미터로 x축 카테고리 순서를 강제 합니다. 나이 순으로 정렬되어 추세를 읽기 쉬워집니다.
25	titanic_df.drop('Age_cat', axis=1, inplace=True)	Age_cat은 시각화 목적으로만 만든 임시 컬럼 이므로 그래프를 그린 뒤 삭제합니다. 학습에는 원본 Age를 사용합니다.

▶ 실행 결과



연령대별·성별 생존율

■ 결과 해석

- 여성은 Child 구간을 제외하면 모든 연령대에서 **생존율이 75% 내외로 높게** 유지됩니다. 특히 Elderly 여성은 100%입니다 (다만 표본이 매우 적어 해석에 주의).
- 남성은 **Baby(약 65%)와 Child(약 43%)에서만 생존율이 높고**, Teenager 이후로는 10~20%대로 급락합니다.
- 이는 "여성과 어린이 먼저"라는 원칙이 남성에게는 **어린 나이에만** 적용되었음을 보여 줍니다.
- **Unknown 구간이 비어 있는** 것은 앞서 설명한 대로 결측 나이를 이미 평균으로 대체했기 때문입니다.
- 결론적으로 **Sex, Age, Pclass**가 생존 예측의 핵심 피처입니다.

4. 레이블 인코딩

```
from sklearn import preprocessing

def encode_features(dataDF):
    features = ['Cabin', 'Sex', 'Embarked']
    for feature in features:
        le = preprocessing.LabelEncoder()
        le = le.fit(dataDF[feature])
        dataDF[feature] = le.transform(dataDF[feature])

    return dataDF

titanic_df = encode_features(titanic_df)
titanic_df.head()
```

줄	코드	상세 설명
3	def encode_features(dataDF):	인코딩 작업을 함수로 묶습니다 . 같은 처리를 테스트 데이터에도 반복 적용하기 위함입니다.
4	features = ['Cabin', 'Sex', 'Embarked']	인코딩할 문자열 컬럼 목록 입니다. Name과 Ticket은 뒤에서 삭제할 예정이라 제외했습니다.
5	for feature in features:	세 컬럼을 차례로 순회 합니다.
6	le = preprocessing.LabelEncoder()	반복문 안에서 매번 새 인코더를 생성 하는 것이 중요합니다. 컬럼마다 고유값이 다르므로 인코더를 공유하면 안 됩니다.
7	le = le.fit(dataDF[feature])	해당 컬럼의 고유값을 학습합니다. fit() 은 자기 자신을 반환하므로 le = 로 다시 받아도 동일합니다.
8	dataDF[feature] = le.transform(dataDF[feature])	변환 결과를 원본 컬럼에 덮어씁니다 . 문자열이 숫자로 대체됩니다.
10	return dataDF	가공된 DataFrame을 반환합니다.
12	titanic_df = encode_features(titanic_df)	함수를 호출해 인코딩을 적용합니다.

▶ 실행 결과

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen...	1	22.0	1	0	A/5 21171	7.2500	7	3
1	2	1	1	Cumings, Mrs. Jo...	0	38.0	1	0	PC 17599	71.2833	2	0
2	3	1	3	Heikkinen, Miss....	0	26.0	0	0	STON/O2. 3101282	7.9250	7	3

3	4	1	1	Futrelle, Mrs. J...	0	35.0	1	0	113803	53.1000	2	3
4	5	0	3	Allen, Mr. Willi...	1	35.0	0	0	373450	8.0500	7	3

■ 결과 해석 - 인코딩 매핑

- **Sex** : female→0, male→1 (알파벳순)
- **Cabin** : A→0, B→1, C→2, D→3, E→4, F→5, G→6, **N→7**, T→8
- **Embarked** : C→0, N→1, Q→2, **S→3**

Name과 Ticket은 아직 문자열로 남아 있습니다. 다음 절에서 삭제됩니다.

5. 전처리 함수 통합 - 재사용 가능한 파이프라인

지금까지의 전처리를 함수로 정리해 하나로 묶습니다. 이렇게 해 두면 새로운 데이터가 들어와도 함수 한 번 호출로 동일한 전처리를 적용할 수 있습니다.

```
from sklearn.preprocessing import LabelEncoder

# Null 처리 함수
def fillna(df):
    df['Age'].fillna(df['Age'].mean(), inplace=True)
    df['Cabin'].fillna('N', inplace=True)
    df['Embarked'].fillna('N', inplace=True)
    df['Fare'].fillna(0, inplace=True)
    return df

# 머신러닝 알고리즘에 불필요한 피처 제거
def drop_features(df):
    df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)
    return df

# 레이블 인코딩 수행.
def format_features(df):
    df['Cabin'] = df['Cabin'].str[:1]
    features = ['Cabin', 'Sex', 'Embarked']
    for feature in features:
        le = LabelEncoder()
        le = le.fit(df[feature])
        df[feature] = le.transform(df[feature])
    return df

# 앞에서 설정한 데이터 전처리 함수 호출
def transform_features(df):
    df = fillna(df)
    df = drop_features(df)
    df = format_features(df)
    return df
```

줄	코드	상세 설명
4~9	def fillna(df):	결측치 처리 전담 함수입니다. Age는 평균, Cabin·Embarked는 'N', Fare는 0으로 대체합니다. 학습 데이터에는 Fare 결측이 없지만, 테스트 데이터에는 있을 수 있으므로 미리 방어 코드를 넣은 것입니다.

13	<code>df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)</code>	예측에 도움이 되지 않는 3개 컬럼을 삭제합니다. • PassengerId : 단순 일련번호 • Name : 891개 모두 고유 • Ticket : 규칙 없는 문자열 이런 식별자 성격의 피처 는 학습에 노이즈만 더합니다.
18	<code>df['Cabin'] = df['Cabin'].str[:1]</code>	선실 번호를 첫 글자(갑판) 로 축약합니다. <code>fillna</code> 에서 'N'으로 채운 뒤 실행되므로 순서가 중요합니다.
19~23	<code>for feature in features: ... LabelEncoder</code>	세 컬럼을 레이블 인코딩합니다.
26~30	<code>def transform_features(df):</code>	세 함수를 순서대로 호출하는 통합 함수 입니다. 순서가 중요합니다: ① 결측 채우기 → ② 불필요 컬럼 제거 → ③ 인코딩 . 결측을 채우기 전에 인코딩하면 NaN 때문에 오류가 납니다.

■ 함수로 묶는 이유

실무에서는 학습 데이터와 테스트 데이터에 **똑같은 전처리**를 적용해야 합니다. 코드를 복사해 두 번 쓰면 **한쪽만 수정하는 실수**가 생기기 쉽습니다. 함수로 만들어 두면 `transform_features(test_df)` 한 줄로 동일한 처리를 보장할 수 있습니다.

⚠ 이 방식의 한계 - LabelEncoder를 재사용하지 않음

`format_features()` 는 호출될 때마다 **새 LabelEncoder를 만들어 fit**합니다. 만약 테스트 데이터의 고유값 구성이 학습 데이터와 다르면 **같은 문자가 다른 숫자로 인코딩**될 수 있습니다(2.5장에서 배운 스케일러 문제와 동일한 구조). 실무에서는 학습 시 fit한 인코더를 **저장했다가 테스트에는 transform만** 적용하거나, `Pipeline` 과 `ColumnTransformer` 를 사용해야 안전합니다.

6. 모델 학습과 예측

6.1 피처와 레이블 분리

```
# 원본 데이터를 재로딩 하고, feature데이터 셋과 Label 데이터 셋 추출.
titanic_df = pd.read_csv('./titanic_train.csv')
y_titanic_df = titanic_df['Survived']
X_titanic_df = titanic_df.drop('Survived', axis=1)

X_titanic_df = transform_features(X_titanic_df)
```

줄	코드	상세 설명
2	<code>titanic_df = pd.read_csv('./titanic_train.csv')</code>	앞에서 여러 번 가공했으므로 원본을 다시 로딩 해 깨끗한 상태에서 시작합니다.
3	<code>y_titanic_df = titanic_df['Survived']</code>	예측 대상인 레이블(y) 을 Series로 추출합니다.
4	<code>X_titanic_df = titanic_df.drop('Survived', axis=1)</code>	Survived를 제외한 나머지를 피처(X) 로 만듭니다. <code>inplace</code> 를 쓰지 않았으므로 원본은 유지됩니다. 레이블이 피처에 섞이면 정답을 보고 예측하는 꼴이 되므로 반드시 분리해야 합니다.
6	<code>X_titanic_df = transform_features(X_titanic_df)</code>	통합 전처리 함수를 피처에만 적용합니다. 결측 처리, 컬럼 삭제, 인코딩이 한 번에 수행됩니다.

▶ 전처리 완료된 X 데이터

```

Pclass Sex Age SibSp Parch Fare Cabin Embarked
0      3   1  22.0     1     0  7.2500     7       3
1      1   0  38.0     1     0 71.2833     2       0
2      3   0  26.0     0     0  7.9250     7       3
3      1   0  35.0     1     0 53.1000     2       3
4      3   1  35.0     0     0  8.0500     7       3

```

```
X_titanic_df.shape: (891, 8)
```

■ 결과 해석

12개였던 컬럼이 **8개로** 줄었습니다(Survived 분리 + 3개 삭제). 모든 값이 숫자이고 결측치가 없으므로 드디어 사이킷런에 입력할 수 있는 상태가 되었습니다.

6.2 학습/테스트 데이터 분리

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X_titanic_df, y_titanic_df, \
                                                test_size=0.2, random_state=11)

```

줄	코드	상세 설명
2	<code>train_test_split(X_titanic_df, y_titanic_df, ...)</code>	891건을 학습 712건 / 테스트 179건(20%)으로 나눕니다. 줄 끝의 <code>\</code> 는 파이썬의 줄 연결 문자로, 한 문장을 여러 줄에 나눠 쓸 때 사용합니다.
3	<code>test_size=0.2, random_state=11</code>	테스트 비율 20%, 시드 11로 재현 가능한 분할을 만듭니다.

▶ 분할 결과

```
X_train.shape: (712, 8)   X_test.shape: (179, 8)
```

6.3 3개 알고리즘 비교

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 결정트리, Random Forest, 로지스틱 회귀를 위한 사이킷런 Classifier 클래스 생성
dt_clf = DecisionTreeClassifier(random_state=11)
rf_clf = RandomForestClassifier(random_state=11)
lr_clf = LogisticRegression(solver='liblinear')

# DecisionTreeClassifier 학습/예측/평가
dt_clf.fit(X_train, y_train)
dt_pred = dt_clf.predict(X_test)
print('DecisionTreeClassifier 정확도: {0:.4f}'.format(accuracy_score(y_test, dt_pred)))

# RandomForestClassifier 학습/예측/평가
rf_clf.fit(X_train, y_train)
rf_pred = rf_clf.predict(X_test)
print('RandomForestClassifier 정확도: {0:.4f}'.format(accuracy_score(y_test, rf_pred)))

# LogisticRegression 학습/예측/평가
lr_clf.fit(X_train, y_train)

```

```
lr_pred = lr_clf.predict(X_test)
print('LogisticRegression 정확도: {0:.4f}'.format(accuracy_score(y_test, lr_pred)))
```

줄	코드	상세 설명
7	dt_clf = DecisionTreeClassifier(random_state=11)	결정 트리 . 데이터를 if-else 규칙으로 나눠 가는 단일 트리 모델입니다.
8	rf_clf = RandomForestClassifier(random_state=11)	랜덤 포레스트 . 여러 개의 결정 트리를 만들어 다수결로 예측 하는 앙상블 기법입니다. 일반적으로 단일 트리보다 성능이 좋습니다.
9	lr_clf = LogisticRegression(solver='liblinear')	로지스틱 회귀 . 이름은 회귀지만 분류 알고리즘 입니다. <code>solver='liblinear'</code> 는 최적화 알고리즘 지정으로, 작은 데이터 세트에 적합 하며 수렴 경고를 방지합니다.
12~14	dt_clf.fit / predict / accuracy_score	학습 → 예측 → 평가 의 3단계입니다. 앞 장에서 배운 흐름과 완전히 동일합니다.
17~19	rf_clf ...	객체 이름만 바뀌었을 뿐 코드 구조가 똑같습니다 . 이것이 사이킷런 Estimator 인터페이스의 힘입니다.
22~24	lr_clf ...	세 번째 알고리즘도 동일한 패턴으로 처리됩니다.

▶ 실행 결과

```
DecisionTreeClassifier 정확도: 0.7877
RandomForestClassifier 정확도: 0.8547
LogisticRegression 정확도: 0.8659
```

■ 결과 해석

알고리즘	정확도	해석
DecisionTree	0.7877	단일 트리는 학습 데이터에 과적합되기 쉬워 성능이 가장 낮습니다.
RandomForest	0.8547	여러 트리의 앙상블 효과 로 과적합이 완화되어 약 7%p 향상되었습니다.
LogisticRegression	0.8659	이 데이터에서는 가장 우수 합니다. 피처가 8개로 적고 관계가 비교적 단순해 선형 모델이 잘 맞았습니다.

복잡한 알고리즘이 항상 좋은 것은 아닙니다. 다만 이 결과는 **단 한 번의 분할**에서 나온 것이므로, 다음 절에서 교차 검증으로 다시 확인합니다.

7. 교차 검증으로 성능 평가

7.1 KFold를 이용한 교차 검증

```
from sklearn.model_selection import KFold

def exec_kfold(clf, folds=5):
    # 폴드 세트를 5개인 KFold객체를 생성, 폴드 수만큼 예측결과 저장을 위한 리스트 객체 생성.
    kfold = KFold(n_splits=folds)
    scores = []

    # KFold 교차 검증 수행.
    for iter_count, (train_index, test_index) in enumerate(kfold.split(X_titanic_df)):
```

```

# X_titanic_df 데이터에서 교차 검증별로 학습과 검증 데이터를 가리키는 index 생성
X_train, X_test = X_titanic_df.values[train_index], X_titanic_df.values[test_index]
y_train, y_test = y_titanic_df.values[train_index], y_titanic_df.values[test_index]

# Classifier 학습, 예측, 정확도 계산
clf.fit(X_train, y_train)
predictions = clf.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
scores.append(accuracy)
print("교차 검증 {0} 정확도: {1:.4f}".format(iter_count, accuracy))

# 5개 fold에서의 평균 정확도 계산.
mean_score = np.mean(scores)
print("평균 정확도: {0:.4f}".format(mean_score))
# exec_kfold 호출
exec_kfold(dt_clf, folds=5)

```

줄	코드	상세 설명
3	def exec_kfold(clf, folds=5):	교차 검증을 수행하는 함수입니다. clf(모델) 을 인자로 받으므로 어떤 알고리즘이든 넣을 수 있습니다. folds=5 는 기본값 이라 생략 가능합니다.
5	kfold = KFold(n_splits=folds)	5개 폴드로 나누는 객체를 생성합니다.
9	for iter_count, (train_index, test_index) in enumerate(kfold.split(X_titanic_df)):	enumerate() 가 반복 횟수(0,1,2...) 를 함께 제공합니다. kfold.split() 이 튜플을 반환하므로 괄호로 묶어 언패킹 합니다.
11~12	X_titanic_df.values[train_index]	.values 가 핵심입니다. X_titanic_df는 DataFrame인데 반환된 인덱스는 위치 기반 이므로, .values 로 ndarray로 변환한 뒤 팬시 인덱싱 해야 합니다. DataFrame에 직접 [train_index] 를 쓰면 컬럼명으로 해석되어 오류가 납니다. (.iloc[train_index] 를 써도 됩니다.)
15~17	clf.fit / predict / accuracy_score	각 폴드마다 새로 학습하고 평가 합니다.
18	scores.append(accuracy)	정확도를 리스트에 누적 합니다.
23	mean_score = np.mean(scores)	5개 폴드의 평균 을 최종 성능으로 삼습니다.
25	exec_kfold(dt_clf, folds=5)	앞에서 만든 결정 트리로 교차 검증을 실행합니다.

▶ 실행 결과

```

교차 검증 0 정확도: 0.7542
교차 검증 1 정확도: 0.7809
교차 검증 2 정확도: 0.7865
교차 검증 3 정확도: 0.7697
교차 검증 4 정확도: 0.8202
평균 정확도: 0.7823

```

■ 결과 해석

폴드별 정확도가 **0.7542~0.8202** 사이에 분포하며 평균은 **0.7823**입니다. 앞서 단일 분할에서 얻은 0.7877과 비슷하지만, **폴드에 따라 최대 6.6%p나 차이**가 난다는 사실이 중요합니다. 단 한 번의 분할 결과만 보고 성능을 판단하면 안 되는 이유입니다.

△ 개선 포인트 - StratifiedKFold 사용 권장

이 코드는 일반 `KFold` 를 사용합니다. 타이타닉 데이터는 생존자 38.4% / 사망자 61.6%로 **클래스가 불균형**하므로, **StratifiedKFold** 를 쓰거나 **cross_val_score** 를 사용하는 것이 더 안정적입니다. `cross_val_score(dt_clf, X_titanic_df, y_titanic_df, cv=5)` 한 줄이면 위 함수 전체를 대체할 수 있으며, Classifier이므로 **Stratified가 자동 적용**됩니다.

7.2 GridSearchCV로 하이퍼 파라미터 튜닝

```
from sklearn.model_selection import GridSearchCV

parameters = {'max_depth':[2,3,5,10],
              'min_samples_split':[2,3,5], 'min_samples_leaf':[1,5,8]}

grid_dclf = GridSearchCV(dt_clf, param_grid=parameters, scoring='accuracy', cv=5)
grid_dclf.fit(X_train, y_train)

print('GridSearchCV 최적 하이퍼 파라미터 :', grid_dclf.best_params_)
print('GridSearchCV 최고 정확도: {0:.4f}'.format(grid_dclf.best_score_))
best_dclf = grid_dclf.best_estimator_

# GridSearchCV의 최적 하이퍼 파라미터로 학습된 Estimator로 예측 및 평가 수행.
dpredictions = best_dclf.predict(X_test)
accuracy = accuracy_score(y_test, dpredictions)
print('테스트 세트에서의 DecisionTreeClassifier 정확도 : {0:.4f}'.format(accuracy))
```

줄	코드	상세 설명
3~4	parameters = {'max_depth':[2,3,5,10], 'min_samples_split':[2,3,5], 'min_samples_leaf':[1,5,8]}	탐색할 하이퍼 파라미터 3종을 정의합니다. • max_depth : 트리 최대 깊이 (4가지) • min_samples_split : 분할에 필요한 최소 샘플 수 (3가지) • min_samples_leaf : 리프 노드가 되기 위한 최소 샘플 수 (3가지) 총 조합은 $4 \times 3 \times 3 = 36$ 가지입니다.
6	GridSearchCV(dt_clf, param_grid=parameters, scoring='accuracy', cv=5)	36개 조합 각각을 5폴드 교차 검증 으로 평가합니다. 총 $36 \times 5 = 180$ 번 학습이 수행됩니다. <code>refit</code> 은 기본값 True라 생략되어 있습니다.
7	grid_dclf.fit(X_train, y_train)	학습 데이터(712건)로만 탐색합니다. 테스트 데이터는 관여하지 않습니다.
9~10	best_params_ / best_score_	최적 조합과 그때의 교차 검증 평균 정확도 를 출력합니다.
11	best_dclf = grid_dclf.best_estimator_	최적 파라미터로 이미 재학습된 모델 을 가져옵니다.
14~16	best_dclf.predict(X_test) / accuracy_score	지금까지 사용하지 않은 테스트 179건 으로 최종 성능을 측정합니다.

▶ 실행 결과

```
GridSearchCV 최적 하이퍼 파라미터 : {'max_depth': 3, 'min_samples_leaf': 5, 'min_samples_split': 2}
GridSearchCV 최고 정확도: 0.7992
테스트 세트에서의 DecisionTreeClassifier 정확도 : 0.8715
```

■ 결과 해석 - 튜닝의 효과

- 기본 설정 결정 트리의 테스트 정확도는 **0.7877**이었는데, 튜닝 후 **0.8715**로 **약 8.4%p 향상**되었습니다. 하이퍼 파라미터 튜닝만으로 얻은 상당한 개선입니다.
- 최적 `max_depth=3` 은 **얕은 트리**입니다. 즉 기본 설정의 깊은 트리가 **과적합**이었고, 깊이를 제한하니 일반화 성능이 좋아진 것입니다.
- `min_samples_leaf=5` 도 같은 맥락으로, 리프에 최소 5개 샘플을 요구해 **과도하게 세밀한 분기를 억제**합니다.
- 튜닝된 결정 트리(0.8715)가 앞의 로지스틱 회귀(0.8659)와 랜덤 포레스트(0.8547)를 **모두 앞질렀습니다**.

⚠ 교차 검증 점수(0.7992) < 테스트 점수(0.8715)인 이유

보통은 교차 검증 점수가 더 높게 나오는데 여기서는 반대입니다. 이는 **테스트 세트가 179건으로 작아** 우연히 쉬운 샘플이 많이 포함되었을 가능성이 큼니다. 표본이 작을수록 이런 변동이 크므로, **단일 수치보다 교차 검증 점수를 더 신뢰**하는 것이 안전합니다.

8. 핵심 요약 및 머신러닝 활용 포인트

8.1 전체 실습 흐름 정리

단계	작업	핵심 코드
①	데이터 로딩·탐색	<code>read_csv()</code> , <code>head()</code> , <code>info()</code>
②	결측치 처리	<code>fillna()</code> — 숫자는 평균, 문자는 'N'
③	피처 가공	<code>Cabin.str[:1]</code> — 갑판 정보만 추출
④	시각화 분석	<code>sns.barplot()</code> — Sex, Pclass, Age가 핵심 요인임을 확인
⑤	불필요 피처 제거	<code>drop(['PassengerId', 'Name', 'Ticket'])</code>
⑥	레이블 인코딩	<code>LabelEncoder</code> — Cabin, Sex, Embarked
⑦	피처/레이블 분리	<code>y = df['Survived']</code> , <code>X = df.drop('Survived', axis=1)</code>
⑧	학습/테스트 분리	<code>train_test_split(test_size=0.2)</code>
⑨	모델 학습·비교	DecisionTree / RandomForest / LogisticRegression
⑩	교차 검증	<code>KFold</code> — 평균 0.7823
⑪	하이퍼 파라미터 튜닝	<code>GridSearchCV</code> — 최종 0.8715

8.2 성능 비교 종합

모델	설정	테스트 정확도
DecisionTreeClassifier	기본값	0.7877
RandomForestClassifier	기본값	0.8547
LogisticRegression	<code>solver='liblinear'</code>	0.8659

DecisionTreeClassifier	GridSearchCV 튜닝 (max_depth=3, min_samples_leaf=5)	0.8715
------------------------	--	--------

8.3 머신러닝 활용 포인트

- **전처리가 성능의 8할** : 이번 실습에서 알고리즘 코드는 몇 줄에 불과했지만, 데이터를 학습 가능한 형태로 만드는 데 대부분의 코드가 쓰였습니다. 실무도 동일합니다.
- **시각화로 가설을 세우기** : barplot 세 개만으로 Sex·Pclass·Age가 핵심 요인임을 파악했습니다. **모델링 전 EDA**가 피처 선택과 엔지니어링의 방향을 정해 줍니다.
- **전처리를 함수로 묶기** : 학습·테스트에 동일한 처리를 보장합니다. 더 나아가 `Pipeline` + `ColumnTransformer` 를 쓰면 **데이터 누수**까지 자동으로 방지됩니다.
- **여러 알고리즘을 먼저 비교** : Estimator 인터페이스가 통일되어 있으므로 비용이 거의 들지 않습니다. 복잡한 모델이 항상 우수한 것은 아닙니다.
- **튜닝의 가치** : 같은 결정 트리가 튜닝만으로 0.7877 → 0.8715가 되었습니다. 알고리즘을 바꾸기 전에 **현재 모델을 제대로 튜닝했는지** 먼저 확인하세요.
- **과적합의 신호** : 최적 max_depth가 3으로 알게 나온 것은 기본 설정이 과적합이었다는 증거입니다. **학습 정확도와 검증 정확도의 격차**를 항상 함께 확인하세요.

⚠ 안정화 버전 참고

- **pandas 2.x** : `df['col'].fillna(값, inplace=True)` 연쇄 할당은 **FutureWarning**이 발생하며 pandas 3.0에서 동작하지 않습니다. `df['col'] = df['col'].fillna(값)` 형태를 사용하세요.
- **scikit-learn 1.7.x** : 본 장에서 사용한 모든 API는 변경 없이 동작합니다.
- **seaborn 0.13+** : `barplot` 에서 `x`, `y` 는 **키워드 인자로 명시**해야 합니다(위치 인자 방식은 제거됨). 본문 코드는 이미 키워드 방식이라 문제없습니다.
- **LogisticRegression** : `solver` 를 지정하지 않으면 기본값 `'lbfgs'` 가 쓰이며, 스케일링하지 않은 데이터에서 **수렴 경고 (ConvergenceWarning)**가 발생할 수 있습니다. `'liblinear'` 지정이나 `StandardScaler` 적용으로 해결됩니다.